Лабораторная работа №9

Архитектура ЭВМ

Альманасра Рами

Content

Цель работы **5**

Задание **6**

Теоретическое введение **7**

Выполнение лабораторной работы **8**

Реализация подпрограмм в NASM 8

Отладка программам с помощью GDB 10

Добавление точек останова 13

Работа с данными программы в GDB 15

Обработка аргументов командной строки в GDB 16

Задание для самостоятельной работы 18

Выводы **23**

Список литературы **24**

Список иллюстраций

1 создание файла 8
2 запкуск программы 8
3 замена программы 8
4 Запуск программы в GDB 11
5 Проверка программы с помощью GDB 11
6 Запуск отладчика с брейкпоинтом 12
7 дисассимилированный код программы 12
8 режим псевдографики 13
9 Список точек останова 14
10 вторая точка останова 14
11 содержимое регистров 15
12 Просмотр содержимого переменных двумя способами 15
13 Изменение содержимого переменных 16
14 значение регистра в различных представлениях 16
15 Подготовка новой программы 17
16 работа стека 17
17 Измененная программа предыдущей лабораторной работы 18
18 запуск программы 18
19 проверка программы в GDB 20
20 новый код программы 21
21 запуск программы 21

Список таблиц

Цель работы

Приобретение навыков написания программ с использованием подпрограмм. Знакомство с методами отладки при помощи GDB и его основными возможностями.

Задание

1. Реализация подпрограмм в NASM
2. Отладка программ с использованием GDB
3. Самостоятельное выполнение заданий по материалам лабораторной работы

Теоретическое введение

Отладка — это процесс поиска и исправления ошибок в программе. В общем случае его можно разделить на четыре этапа: - обнаружение ошибки; - поиск её местонахождения; - определение причины ошибки; - исправление ошибки.

Можно выделить следующие типы ошибок:

• синтаксические ошибки — обнаруживаются во время трансляции исходного кода и вызваны нарушением ожидаемой формы или структуры языка; • семантические ошибки — являются логическими и приводят к тому, что программа запускается, отрабатывает, но не даёт желаемого результата; • ошибки в процессе выполнения — не обнаруживаются при трансляции и вызывают прерывание выполнения программы (например, это ошибки, связанные с переполнением или делением на ноль).

Второй этап — поиск местонахождения ошибки. Некоторые ошибки обнаружить довольно трудно. Лучший способ найти место в программе, где находится ошибка, это разбить программу на части и произвести их отладку отдельно друг от друга. Третий этап — выяснение причины ошибки. После определения местонахождения ошибки обычно проще определить причину неправильной работы программы. Последний этап — исправление ошибки. После этого при повторном запуске программы, может обнаружиться следующая ошибка, и процесс отладки начнётся заново.

Выполнение лабораторной работы

Реализация подпрограмм в NASM

Создаю файл lab09-1.asm в каталог для выполнения лабораторной работы № 9 : (Figure 1).

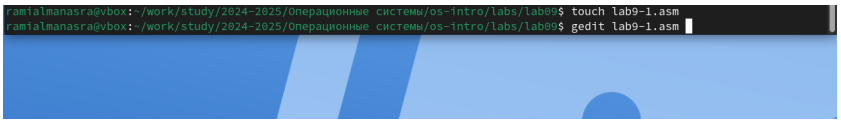

Рис. 1: создание файла

Я копирую код из листинга в файл, компилирую и запускаю его. Эта программа выполняет вычисление функции 2x+7 (Figure 2).

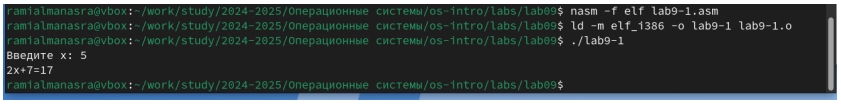

Рис. 2: запкуск программы

Изменяю текст программы, добавляя в него подпрограмму. Теперь она вычисляет значение функции для выражения f(g(x)) (Figure 3).

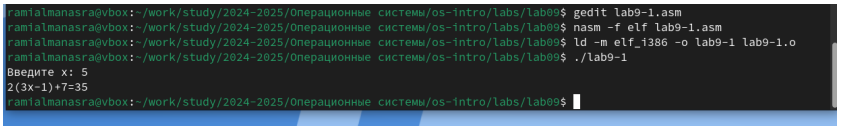

Рис. 3: замена программы

новый код программы:

```
%include 'in_out.asm'
SECTION .data
msg: DB 'Введите x: ',0
result: DB '2(3x-1)+7=', 0

SECTION .bss
x: RESB 80
res: RESB 80

SECTION .text
GLOBAL _start
_start:
mov eax, msg
call sprint

mov ecx, x
mov edx, 80
call sread

mov eax, x
call atoi

call _calcul

mov eax, result
call sprint
mov eax, [res]
call iprintLF
```

```
call quit

_calcul:
push eax
call _subcalcul

mov ebx, 2
mul ebx
add eax, 7

mov [res], eax
pop eax
ret

_subcalcul:
mov ebx, 3
mul ebx
sub eax, 1
ret
```

Отладка программам с помощью GDB

Копирую программу из второго листинга в созданный файл, и провожу трансляцию с ключом -g (Figure 4).

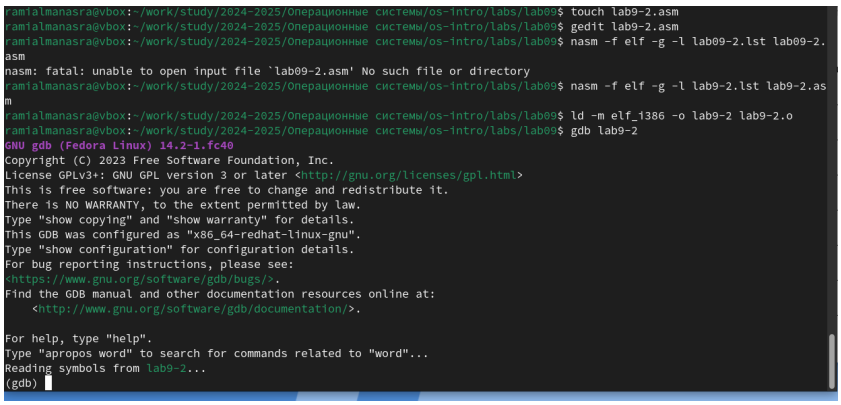

Рис. 4: Запуск программы в GDB

Запустив программу командой run, я убедился, что она работает корректно (Figure 5).

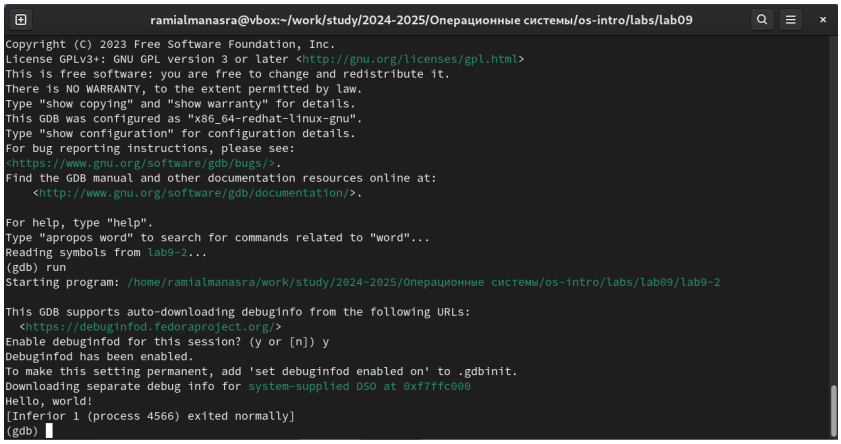

Рис. 5: Проверка программы с помощью GDB

Для более подробного анализа программы установлю брейкпоинт на метку _start (Figure 6).

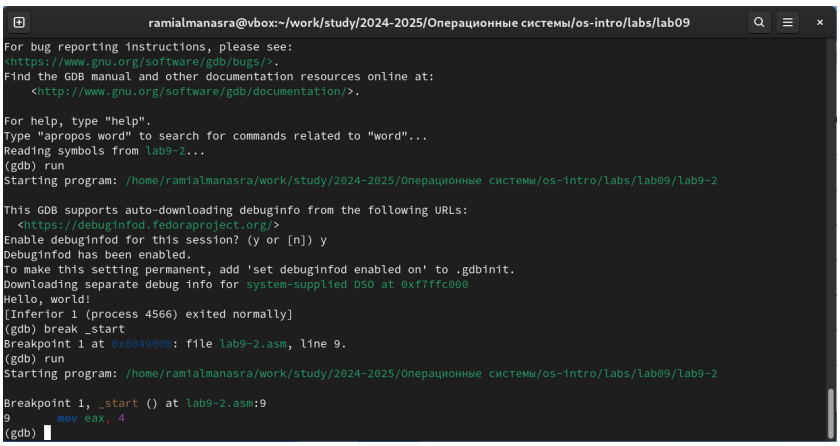

Рис. 6: Запуск отладчика с брейкпоинтом

Далее я смотрю на дизассемблированный код программы, переведенный в команду с синтаксисом Intel (Figure 7).

Различия между синтаксисом ATT и Intel заключаются в порядке следования операндов (ATT: сначала исходный операнд; Intel: сначала целевой операнд), их размере (ATT: явно задается с помощью суффиксов; Intel: неявно определяется контекстом) и именах регистров (ATT: перед ‘%’; Intel: без префиксы).

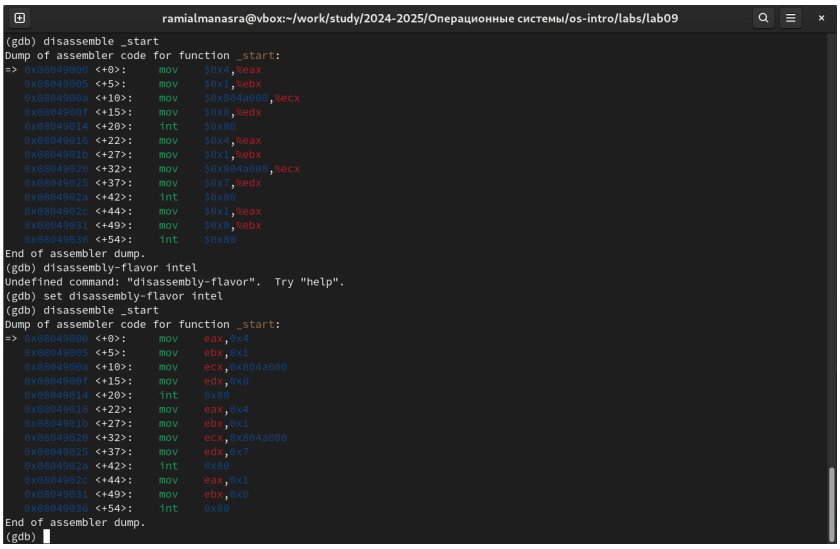

Рис. 7: дисассимилированный код программы

Включаю режим псевдографики для более удобного анализа программы

(Figure 8).

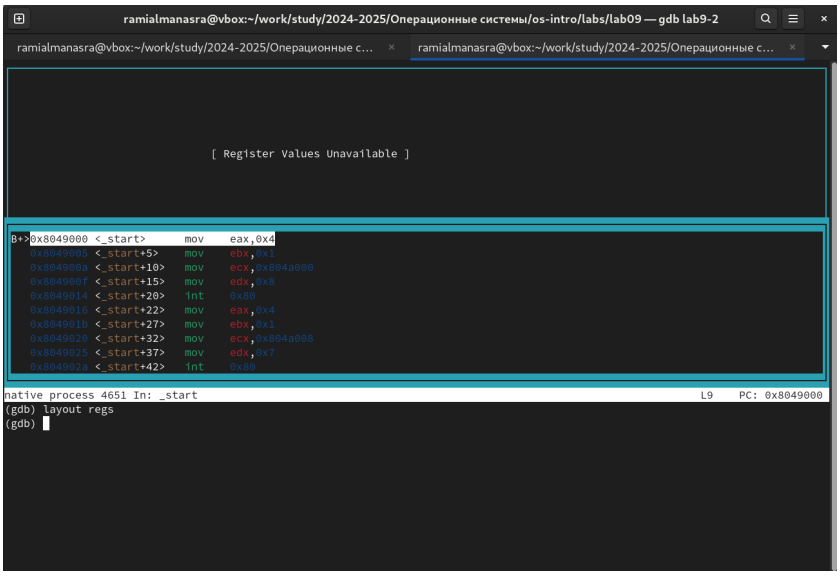

Рис. 8: режим псевдографики

Добавление точек останова

Я проверяю в псевдографическом режиме, сохранена ли точка останова (Figure 9).

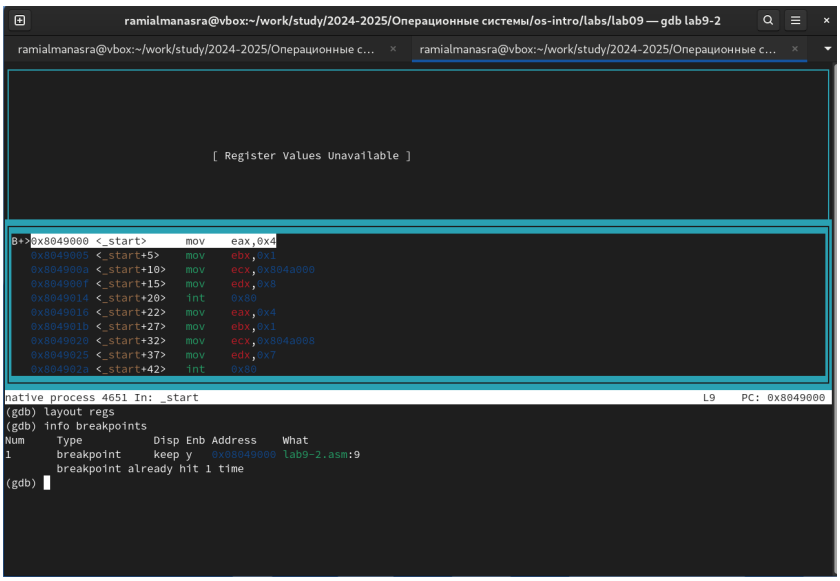

Рис. 9: Список точек останова

Установлю еще одну точку останова по адресу инструкции и смотрю информацию о всех установленных точках останова (Figure 10).

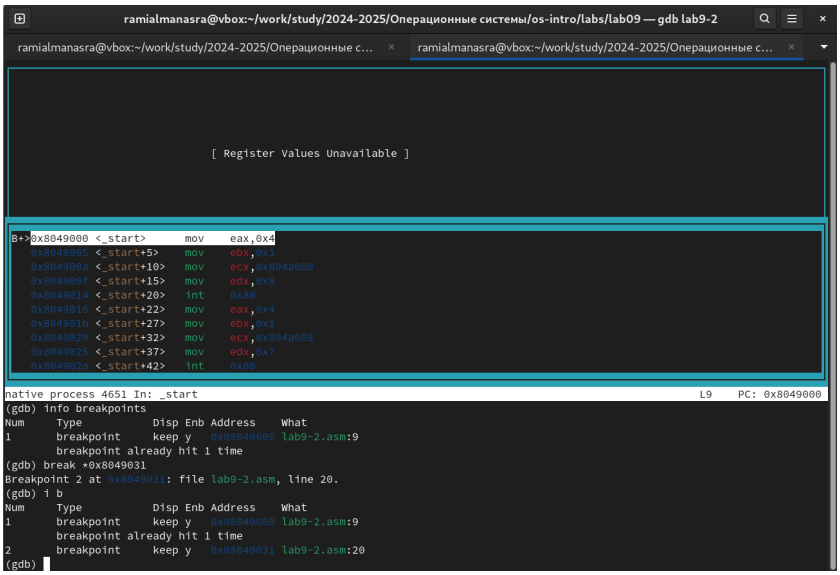

Рис. 10: вторая точка останова

Работа с данными программы в GDB

Я просматриваю содержимое регистров, используя команду `info registers` (Figure 11).

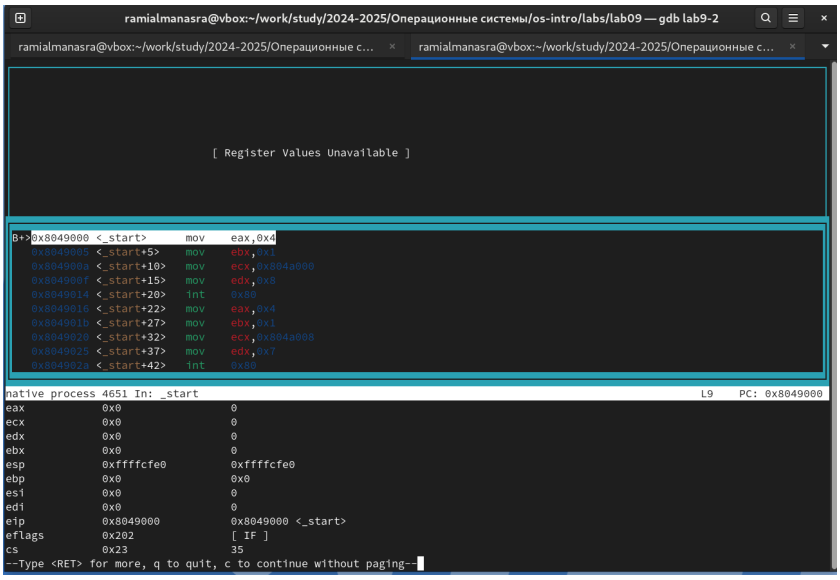

Рис. 11: содержимое регистров

Я просматриваю содержимое переменных по имени и адресу (Figure 12).

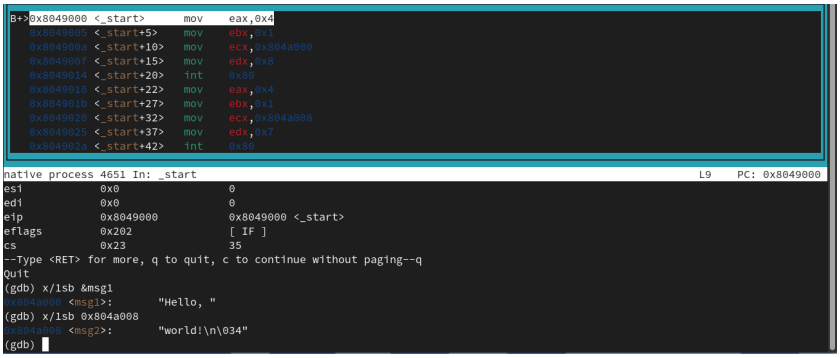

Рис. 12: Просмотр содержимого переменных двумя способами

изменяю содержимое переменных(Figure 13).

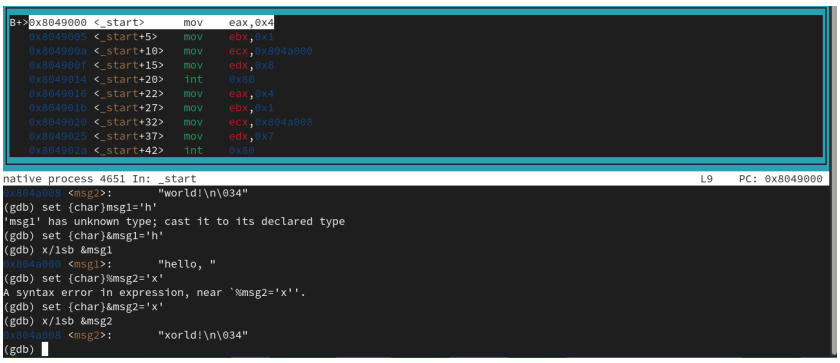

Рис. 13: Изменение содержимого переменных

Я вывожу значение регистра edx в различных форматах и Используя команду “set”, изменяю содержимое регистра “ebx” (Figure 14).

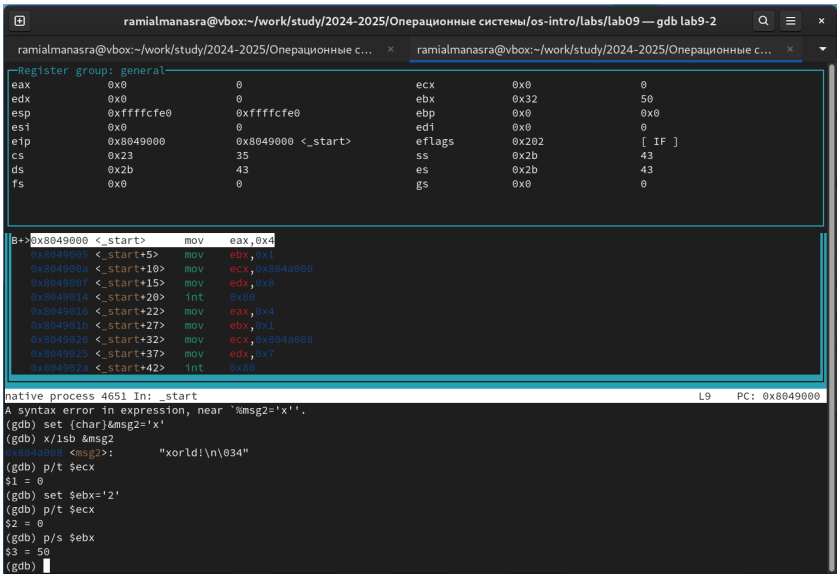

Рис. 14: значение регистра в различных представлениях

Обработка аргументов командной строки в GDB

копирую программу из предыдущей лабораторной работы в текущий каталог и создаю исполняемый файл со списком и файлом отладки (Figure 16).

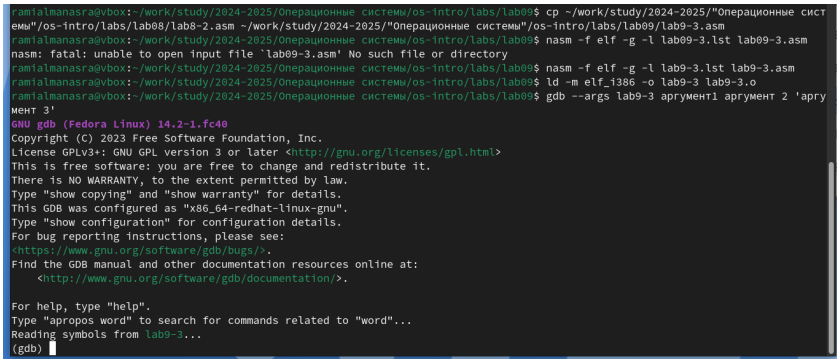

Рис. 15: Подготовка новой программы

Я запускаю программу в режиме отладки, задавая аргументы, указываю точку останова и начинаю отладку. Я проверяю работу стека, изменяя аргумент команды для просмотра регистра esp на +4 (число определяется разрядностью системы, а указатель void занимает 4 байта); ошибка с аргументом +24 означает, что входные аргументы программы закончились. (Figure 17).

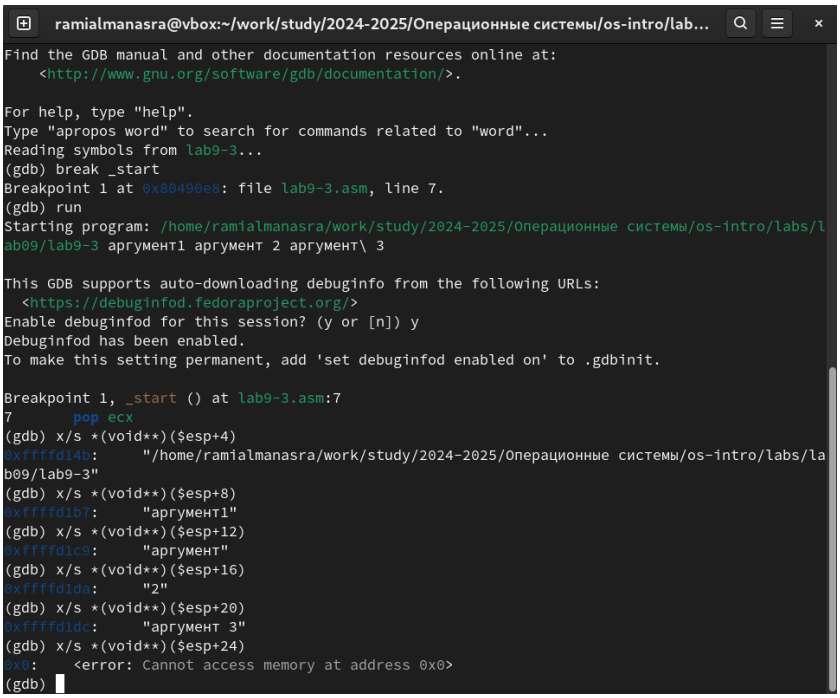

Рис. 16: работа стека

Задание для самостоятельной работы

1. изменяю программу самостоятельной части предыдущей лабораторной работы с помощью подпрограммы (Figure 18).

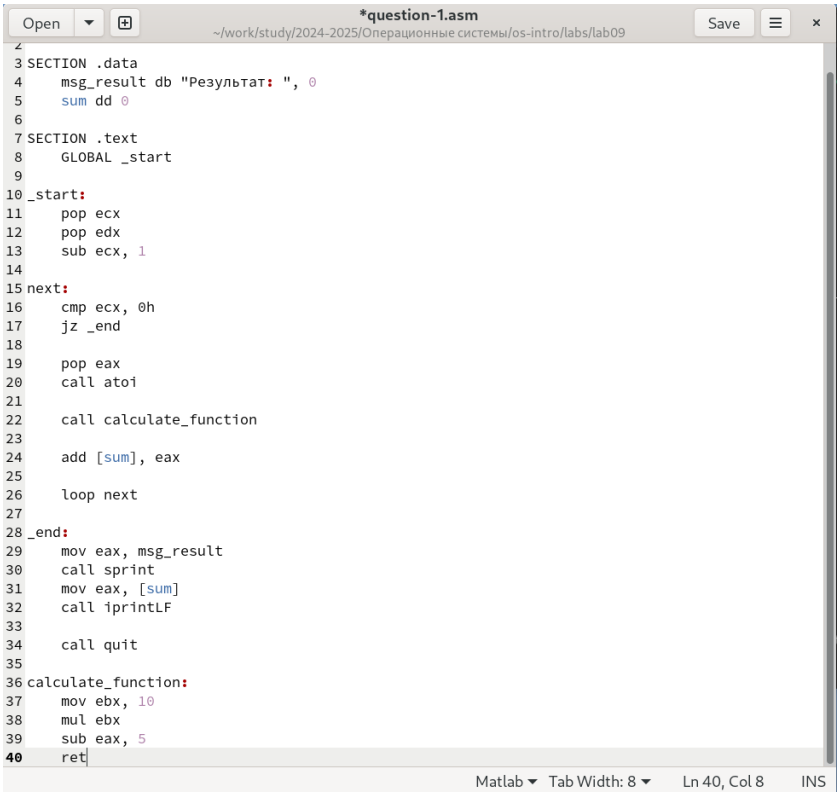

Рис. 17: Измененная программа предыдущей лабораторной работы

Запускаю программу и проверяю что работает (Figure 19).

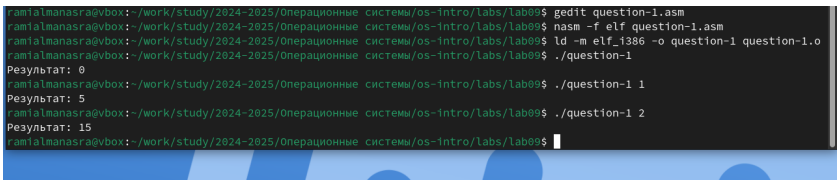

Рис. 18: запуск программы

код программы:

```
%include 'in_out.asm'

SECTION .data
    msg_result db "Результат: ", 0
    sum dd 0

SECTION .text
    GLOBAL _start

_start:
    pop ecx
    pop edx
    sub ecx, 1

next:
    cmp ecx, 0h
    jz _end

    pop eax
    call atoi

    call calculate_function

    add [sum], eax

    loop next

_end:
    mov eax, msg_result
```

```
    call sprint
    mov eax, [sum]
    call iprintLF

    call quit

calculate_function:
    mov ebx, 10
    mul ebx
    sub eax, 5
    ret
```

2. Проверяю программу,С помощью отладчика GDB, анализируя изменения значений регистров. (Figure 20). Результат (3 + 2) сохраняется в ebx, но mul использует eax вместо ebx. Из-за этого результат неверный.

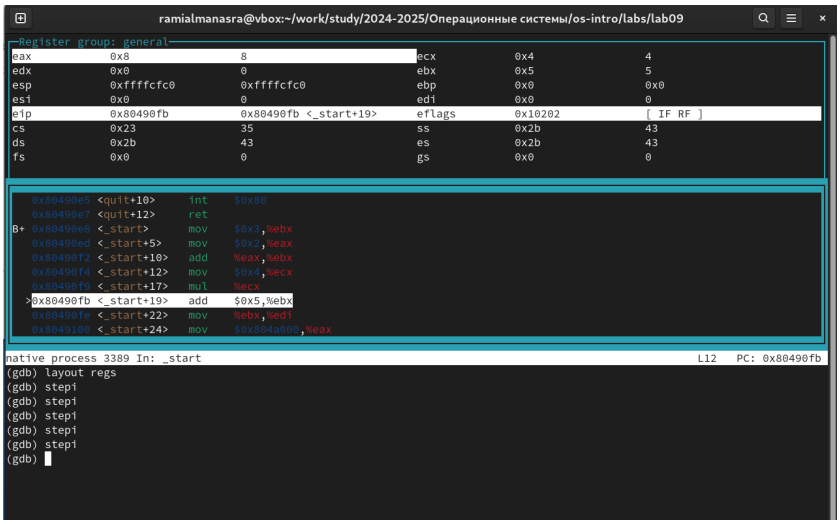

Рис. 19: проверка программы в GDB

исправляю код программы (Figure 21).

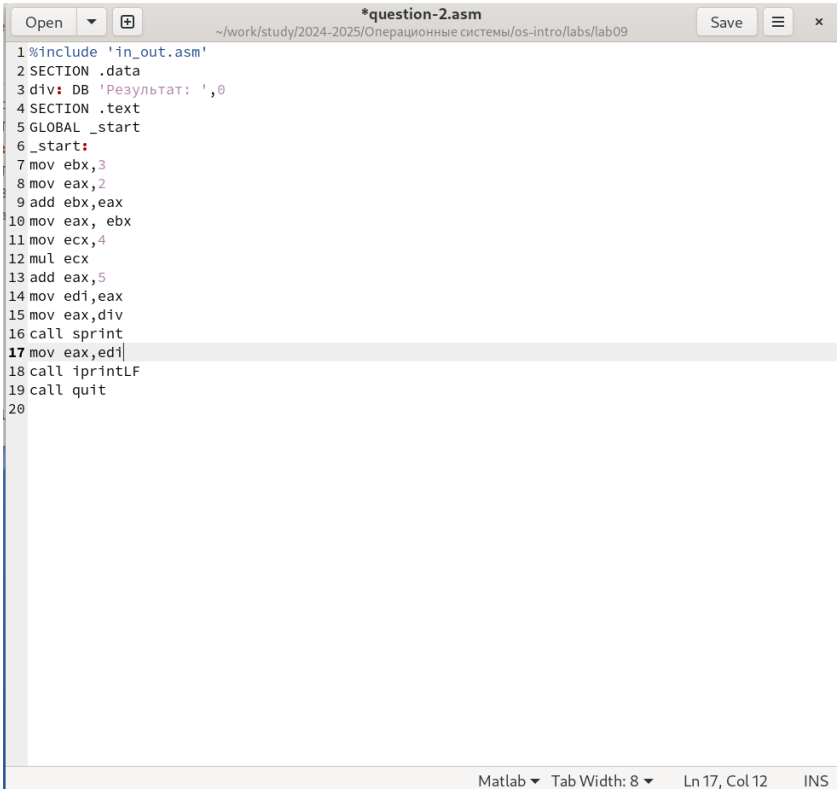

Рис. 20: новый код программы

Теперь программа дает правильный результат (Figure 22).

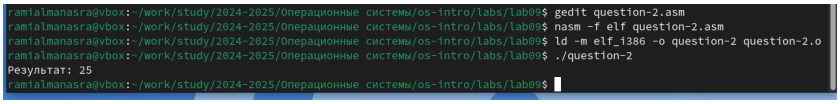

Рис. 21: запуск программы

исправленный код:

```
%include 'in_out.asm'
SECTION .data
div: DB 'Результат: ',0
SECTION .text
GLOBAL _start
_start:
```

```
mov ebx,3
mov eax,2
add ebx,eax
mov eax, ebx
mov ecx,4
mul ecx
add eax,5
mov edi,eax
mov eax,div
call sprint
mov eax,edi
call iprintLF
call quit
```

Выводы

В результате выполнения этой лабораторной работы я приобрел навыки написания программ с использованием подпрограмм, а также познакомился с методами отладки с использованием GDB и его основными возможностями.

Список литературы

1. Course on TUIS
2. Laboratory work No. 9
3. Programming in NASM Assembly Language Stolyarov A. V.